🦴 🪨 🦕 🔬 🌿

Fossilyte

Fossil Collection Storage SaaS

Product	Fossilyte
Document Type	Software Design Document (SDD)
Status	**ACTIVE Personal Project**
Version	1.0.0
Author	Solo Engineer / Owner
Date	April 2026
Deployment	Google Cloud Platform (GCP) Cloud Run (Docker)
Database	NeonDB (Serverless PostgreSQL)
ML	Mistral AI Fossil Image Classification
Funding Model	**Self-funded passion project personal expense**

TypeScript · Next.js · Tailwind CSS · NeonDB · Docker · GCP · Mistral AI

1. Executive Summary

Fossilyte is a self-funded Software-as-a-Service application built for casual fossil collectors an audience that has been consistently overlooked by existing collection management tools, which either target professional paleontologists with overly complex interfaces or cater to general collectibles with no domain-specific features.

The application provides a clean, intuitive platform for collectors to catalogue their fossil specimens, manage high-quality images, browse their entire collection through a visual dashboard, and leverage AI-powered fossil identification from photographs. It fills a clear gap: no purpose-built SaaS for the casual fossil collector community existed before this project.

Pillar	Solution
Market Gap	First purpose-built SaaS for casual fossil collectors; no comparable product exists in the market
Core Experience	Add, edit, remove fossil entries with photo upload; rich dashboard gallery view
AI Identification	Mistral AI multimodal model classifies fossil type from uploaded photograph with confidence score
Infrastructure	Dockerized Next.js app on GCP Cloud Run; NeonDB serverless PostgreSQL; GCS image storage
Cost Model	Personal expense designed for minimal operational cost with serverless-first architecture

2. Problem Statement

2.1 The Market Gap

Fossil collecting is a popular and growing hobby. Thousands of casual collectors weekend diggers, estate sale hunters, gift recipients, and nature enthusiasts accumulate meaningful personal collections with no structured way to catalogue, document, or identify their finds.

Existing tooling falls into two unusable categories:

- Professional paleontology databases (PBDB, Specify, EMu) are designed for institutions and researchers, with steep learning curves and feature sets irrelevant to hobbyists.
- General collectibles apps (Collector, Collectify) have no fossil-specific taxonomy, metadata fields, or AI identification capabilities.
- Spreadsheet workarounds are the de facto solution brittle, not mobile-friendly, and incapable of image-centric browsing or AI enrichment.

2.2 Pain Points for Casual Collectors

Pain Point	Severity	Current Workaround
No fossil-specific metadata fields	HIGH	Generic notes fields in spreadsheets
No image-first collection browser	HIGH	Photo albums with no data linkage
Cannot identify unknown specimens	HIGH	Manual forum posts, guesswork
No cloud backup for collection data	MEDIUM	Local files, at risk of loss
No shareable collection view	MEDIUM	Screenshots shared via social media
No mobile-friendly interface	MEDIUM	Pinch-zoom desktop spreadsheets

3. Goals and Non-Goals

3.1 Goals

- Provide a delightful, image-first web application for casual fossil collectors to catalogue their specimens.
- Support complete CRUD operations for fossil entries: name, species, period, location found, acquisition date, notes, and one or more photographs.
- Deliver a visual dashboard gallery that showcases the entire collection at a glance, with filtering and search.
- Integrate Mistral AI multimodal identification so collectors can upload a photo and receive an AI-generated fossil type classification with confidence indicators.
- Deploy as a fully containerized application on GCP with minimal operational cost suitable for a self-funded personal project.
- Achieve sufficient scalability to serve hundreds to low thousands of concurrent users without architectural change.
- Maintain 99.5%+ availability using serverless and managed infrastructure components.

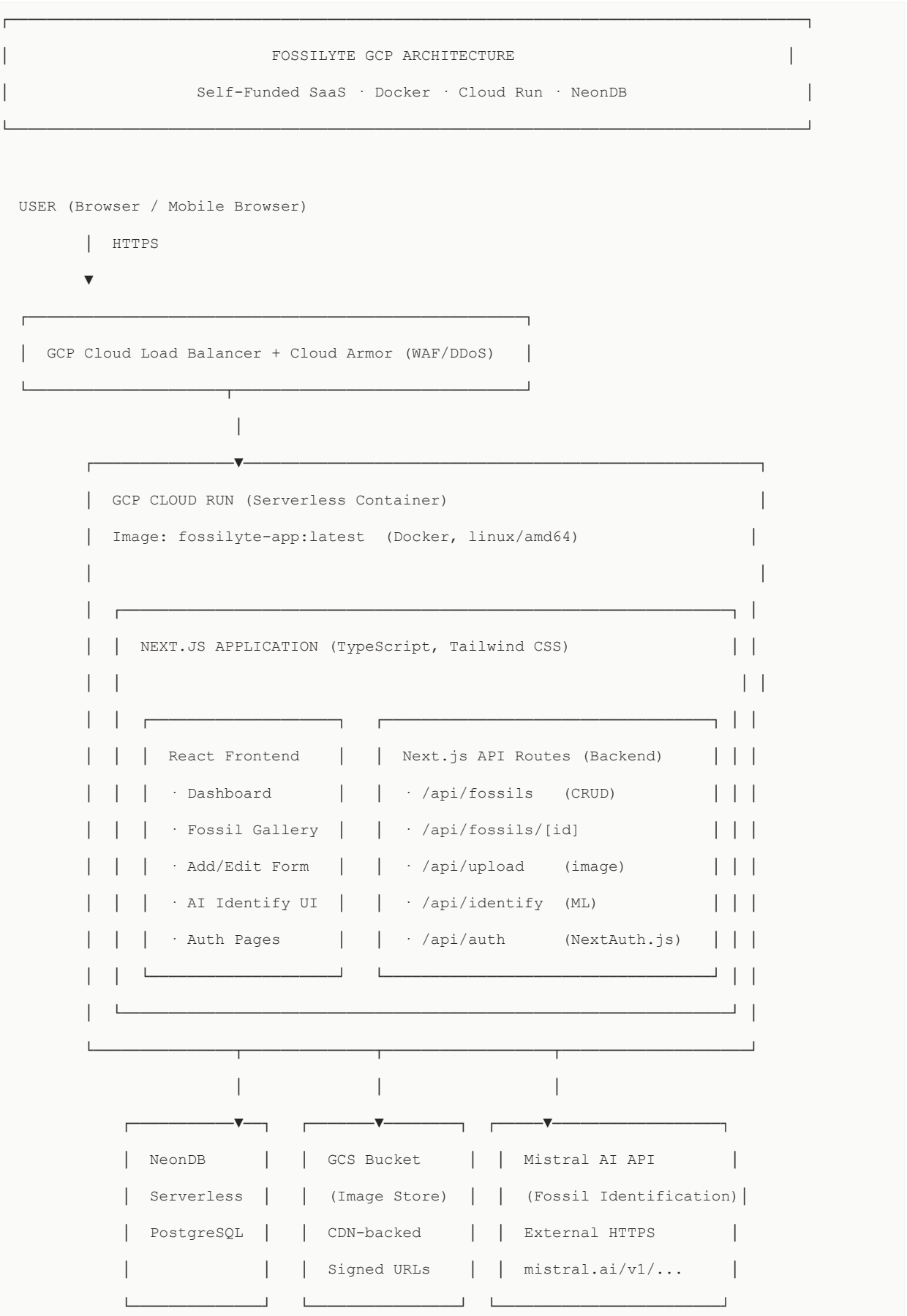
3.2 Non-Goals

- Fossilyte is not a professional paleontology research platform no stratigraphic data layers, specimen loan tracking, or institution-grade taxonomic hierarchies.
- No e-commerce or marketplace functionality buying and selling fossils is out of scope.
- No native mobile applications (iOS/Android) in v1 the web application is responsive and mobile-compatible.
- No multi-user collection collaboration or organization accounts in v1 single-user personal collections only.
- No real-time features (live notifications, multiplayer) not justified by the use case or the project's cost envelope.

4. System Architecture

4.1 High-Level Architecture

Fossilyte is built as a monolithic Next.js application following a server-component-first pattern. The same TypeScript codebase serves the React frontend, API routes (backend), and server-side rendering. The application is containerized with Docker and deployed to GCP Cloud Run for serverless container execution. All persistent data lives in NeonDB (serverless PostgreSQL), image files in Google Cloud Storage, and ML inference is delegated to the Mistral AI API.


FOSSILYTE GCP ARCHITECTURE
Self-Funded SaaS · Docker · Cloud Run · NeonDB
USER (Browser / Mobile Browser)
HTTPS
GCP Cloud Load Balancer + Cloud Armor (WAF/DDoS)
GCP CLOUD RUN (Serverless Container)
Image: fossilyte-app:latest (Docker, linux/amd64)
NEXT.JS APPLICATION (TypeScript, Tailwind CSS)
React Frontend
· Dashboard
· Fossil Gallery
· Add/Edit Form
· AI Identify UI
· Auth Pages
Next.js API Routes (Backend)
· /api/fossils (CRUD)
· /api/fossils/[id]
· /api/upload (image)
· /api/identify (ML)
· /api/auth (NextAuth.js)
NeonDB
Serverless
PostgreSQL
GCS Bucket
(Image Store)
CDN-backed
Signed URLs
Mistral AI API
(Fossil Identification)
External HTTPS
mistral.ai/v1/...

4.2 Infrastructure Components

Component	Technology	Role
Frontend	Next.js 14 + React	Server components, React UI, Tailwind CSS styling, SSR/SSG
Backend API	Next.js API Routes	REST endpoints for fossil CRUD, image upload, ML trigger; runs in same container
Language	TypeScript	End-to-end type safety across frontend, backend, and database query layer
Styling	Tailwind CSS	Utility-first CSS; responsive grid for fossil gallery dashboard
Database	NeonDB (PostgreSQL)	Serverless autoscaling PostgreSQL; scales to zero on inactivity ideal for personal project cost management
ORM	Drizzle ORM	Type-safe SQL query builder for TypeScript; zero-overhead schema migrations
Auth	NextAuth.js	Session-based auth; supports email/password and OAuth (Google); JWT sessions
Image Storage	GCS (Cloud Storage)	Fossil photograph storage; served via CDN with time-limited signed URLs
ML / AI	Mistral AI API	Multimodal model (pixtral-12b) for fossil image classification and species identification
Containerization	Docker	Multi-stage Dockerfile; linux/amd64 build for Cloud Run compatibility
Deployment	GCP Cloud Run	Serverless container platform; auto-scales 0→N instances; pay-per-request billing
CI/CD	GCP Cloud Build	Trigger on git push: build Docker image → push to Artifact Registry → deploy to Cloud Run
Image Registry	GCP Artifact Registry	Docker image storage; vulnerability scanning; access via IAM
Secrets	GCP Secret Manager	DB connection strings, Mistral API key, NextAuth secret never in environment files
Observability	GCP Cloud Logging	Structured application logs from Cloud Run; error alerting via log-based metrics
WAF / DDoS	GCP Cloud Armor	OWASP rule set; rate limiting; protects the Cloud Run service via Load Balancer

5. Module Design Specifications

5.1 Fossil Management Module (Add / Edit / Remove)

The fossil management module is the core data-entry surface. It allows authenticated collectors to create, update, and delete fossil entries. Each entry is a structured record with domain-specific metadata fields and up to 5 associated photographs stored in GCS.

5.1.1 Data Schema

```
-- fossils table (NeonDB / PostgreSQL)
CREATE TABLE fossils (
  id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id         UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  name            TEXT NOT NULL,               -- Collector's display name
  common_name     TEXT,                        -- e.g. "Megalodon Tooth"
  scientific_name TEXT,                        -- e.g. "Otodus megalodon"
  geological_period TEXT,                      -- e.g. "Cretaceous"
  formation       TEXT,                        -- e.g. "Niobrara Formation"
  location_found  TEXT,                        -- Free-text location
  acquisition_date DATE,
  acquisition_method TEXT,                     -- DIG | PURCHASED | GIFTED | TRADED
  condition       TEXT,                        -- EXCELLENT | GOOD | FAIR | POOR
  size_mm         INTEGER,                     -- Longest dimension in mm
  weight_g        NUMERIC(8,2),                -- Weight in grams
  notes           TEXT,                        -- Free-form collector notes
  ai_identified   BOOLEAN DEFAULT false,
  ai_label        TEXT,                        -- AI classification result
  ai_confidence   NUMERIC(4,3),                -- 0.000 - 1.000
  created_at      TIMESTAMPTZ DEFAULT now(),
  updated_at      TIMESTAMPTZ DEFAULT now()
);

CREATE TABLE fossil_images (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  fossil_id   UUID NOT NULL REFERENCES fossils(id) ON DELETE CASCADE,
  gcs_key     TEXT NOT NULL,   -- GCS object path
  is_primary  BOOLEAN DEFAULT false,
  uploaded_at TIMESTAMPTZ DEFAULT now()
);

CREATE INDEX idx_fossils_user_id ON fossils(user_id);
CREATE INDEX idx_fossil_images_fossil_id ON fossil_images(fossil_id);
```

5.1.2 API Endpoints

Method + Path	Auth	Description
GET /api/fossils	**Required**	Return paginated list of authenticated user's fossils with primary image URLs
POST /api/fossils	**Required**	Create new fossil entry; returns created record with generated id
GET /api/fossils/[id]	**Required**	Return single fossil detail with all images; ownership enforced
PATCH /api/fossils/[id]	**Required**	Partial update of fossil metadata; validates ownership before write
DELETE /api/fossils/[id]	**Required**	Delete fossil record and all associated GCS images; cascade deletes images table rows
POST /api/upload	**Required**	Upload fossil image to GCS; returns gcs_key and signed URL; max 10 MB per image
DELETE /api/upload/[key]	**Required**	Remove single image from GCS and fossil_images record

5.2 Dashboard Module

The dashboard is the primary landing screen for authenticated users. It presents the collector's fossil collection as a responsive image grid a visual-first experience that mirrors how collectors think about their finds. The dashboard supports real-time filtering, text search, sorting, and a toggle between gallery and list views.

5.2.1 Dashboard Features

Feature	Implementation Detail
Image Gallery Grid	CSS Grid with Tailwind; 2-col mobile, 3-col tablet, 4-col desktop; card shows primary photo, name, period
Search	Debounced client-side text search across name, common_name, scientific_name, location_found, notes
Filter Panel	Filter by geological period, acquisition method, condition, AI-identified flag; multi-select dropdowns
Sort Options	Sort by date added (newest/oldest), name (A–Z), geological period, size, acquisition date
List / Grid Toggle	Grid view for visual browsing; List view for data-dense comparison with all metadata columns
Collection Stats	Header stats bar: total specimens, unique periods, total estimated value (if user adds values), AI-identified count
Empty State	Illustrated onboarding prompt for new users with single CTA to add first fossil
Infinite Scroll	Loads 24 fossils per page; Intersection Observer triggers next page load; no pagination buttons

5.3 ML Identification Module (Mistral AI)

The AI identification module is the most distinctive feature of Fossilyte. A collector can upload or select an existing photo and receive an AI-generated classification of the fossil type, estimated geological period, and confidence score. The identification is powered by Mistral's Pixtral-12B multimodal model via the Mistral AI REST API.

5.3.1 Identification Flow

```
User Action: Upload photo or select existing fossil image
       |
       ▼
POST /api/identify
{ fossil_id: "uuid", image_key: "gcs/path/to/image.jpg" }
       |
       ▼
API Route Handler:
1. Validate session reject if unauthenticated
2. Validate fossil ownership (fossil.user_id === session.user.id)
3. Generate GCS signed URL for image (5-minute expiry)
4. Construct Mistral prompt:
   System: "You are a paleontology expert. Analyze the fossil image.
            Return JSON: { fossil_type, common_name, scientific_name,
            geological_period, confidence (0-1), reasoning }"
   User:   [image_url from signed URL]
       |
       ▼
Mistral API (pixtral-12b):
POST https://api.mistral.ai/v1/chat/completions
{ model: "pixtral-12b-2409",
  messages: [{ role: "user", content: [{ type: "image_url", ... }, { type: "text", ... }] }],
  response_format: { type: "json_object" }
}
       |
       ▼
Parse response JSON → validate schema
UPDATE fossils SET ai_label=..., ai_confidence=..., ai_identified=true WHERE id=...
       |
       ▼
Return to client:
{ fossil_type, common_name, scientific_name, geological_period, confidence, reasoning }
       |
       ▼
UI: Render identification card with confidence meter
    Offer "Apply to record" button to save AI labels to fossil entry
```

5.3.2 AI Response Schema

```
interface MistralFossilResponse {
  fossil_type:       string;   // e.g. "Shark Tooth"
  common_name:       string;   // e.g. "Megalodon Tooth"
  scientific_name:   string;   // e.g. "Otodus megalodon"
  geological_period: string;   // e.g. "Miocene (23-5.3 Ma)"
  confidence:        number;   // 0.0 - 1.0
  reasoning:         string;   // Human-readable identification rationale
}
```

6. Data Flow Diagrams

6.1 User Authentication Flow

```
User → /login page
       |  Submit email + password (or Google OAuth button)
       ▼
NextAuth.js POST /api/auth/signin
       |  Credentials provider: hash compare vs users.password_hash (bcrypt)
       |  OAuth provider: Google OAuth2 → creates/links user record
       ▼
NeonDB: SELECT * FROM users WHERE email = $1
       |  Password verified → session created
       ▼
JWT session cookie set (httpOnly, Secure, SameSite=Strict)
Redirect → /dashboard
```

6.2 Add Fossil Flow

```
User → /fossils/new (Add Fossil form)
       │
       ├─ Step 1: Fill metadata (name, period, location, notes, etc.)
       │
       ├─ Step 2: Upload images
       │          POST /api/upload (multipart/form-data)
       │          │  Server: validate MIME type (jpg/png/webp), size ≤ 10MB
       │          │  Upload to GCS: fossils/{user_id}/{uuid}.{ext}
       │          │  Return: { gcs_key, signed_url }
       │          UI: show image preview with signed_url
       │
       ├─ Step 3: (Optional) Click "AI Identify" on uploaded photo
       │          POST /api/identify → Mistral → display result
       │          User clicks "Apply" → pre-fills form fields
       │
       └─ Step 4: Submit form
                  POST /api/fossils
                  { name, period, location, ..., image_keys: [gcs_key...] }
                  │
                  NeonDB transaction:
                  ├─ INSERT INTO fossils (...) RETURNING id
                  └─ INSERT INTO fossil_images (fossil_id, gcs_key, is_primary)
                  │
                  Redirect → /fossils/{id} (detail page)
```

6.3 Dashboard Load Flow

```
User → /dashboard
       │  Next.js Server Component (SSR)
       ▼
getServerSideSession() → validate JWT → get user_id
       │
       ▼
NeonDB: SELECT f.*, fi.gcs_key as primary_image
        FROM fossils f
        LEFT JOIN fossil_images fi ON fi.fossil_id = f.id AND fi.is_primary = true
        WHERE f.user_id = $1
        ORDER BY f.created_at DESC
        LIMIT 24 OFFSET $2
       │
       ▼
For each primary_image gcs_key:
Generate GCS signed URL (1-hour expiry)
       │
       ▼
Render: <FossilGallery fossils={data} /> React Server Component
Client: Infinite scroll → fetch /api/fossils?page=N on scroll trigger
Client: Search/filter → debounced fetch /api/fossils?q=&period=
```

7. Docker & GCP Deployment

7.1 Dockerfile (Multi-Stage Build)

```
# ── Stage 1: Dependencies ──────────────────────────────────────
FROM node:20-alpine AS deps
WORKDIR /app
COPY package.json pnpm-lock.yaml ./
RUN corepack enable && pnpm install --frozen-lockfile

# ── Stage 2: Build ─────────────────────────────────────────────
FROM node:20-alpine AS builder
WORKDIR /app
COPY --from=deps /app/node_modules ./node_modules
COPY . .
RUN pnpm build            # next build → .next/standalone

# ── Stage 3: Production Runtime ────────────────────────────────
FROM node:20-alpine AS runner
WORKDIR /app
ENV NODE_ENV=production
RUN addgroup -g 1001 -S nodejs && adduser -S nextjs -u 1001
COPY --from=builder --chown=nextjs:nodejs /app/.next/standalone ./
COPY --from=builder --chown=nextjs:nodejs /app/.next/static ./.next/static
COPY --from=builder /app/public ./public
USER nextjs
EXPOSE 3000
CMD ["node", "server.js"]

# Image size target: < 250 MB   Production runtime: ~90 MB
```

7.2 Cloud Build CI/CD Pipeline

```
# cloudbuild.yaml

steps:

  - name: gcr.io/cloud-builders/docker

    args: [

      "build",

      "--platform", "linux/amd64",

      "-t", "us-central1-docker.pkg.dev/$PROJECT_ID/fossilyte/app:$COMMIT_SHA",

      "."

    ]


  - name: gcr.io/cloud-builders/docker

    args: ["push", "us-central1-docker.pkg.dev/$PROJECT_ID/fossilyte/app:$COMMIT_SHA"]


  - name: gcr.io/google.com/cloudsdktool/cloud-sdk

    entrypoint: gcloud

    args: [

      "run", "deploy", "fossilyte",

      "--image", "us-central1-docker.pkg.dev/$PROJECT_ID/fossilyte/app:$COMMIT_SHA",

      "--region", "us-central1",

      "--platform", "managed",

      "--allow-unauthenticated",

      "--memory", "512Mi",

      "--cpu", "1",

      "--min-instances", "0",

      "--max-instances", "10",

      "--set-secrets",

      "DATABASE_URL=fossilyte-db-url:latest,MISTRAL_API_KEY=mistral-
key:latest,NEXTAUTH_SECRET=nextauth-secret:latest"

    ]
```

7.3 GCP Services Configuration

GCP Service	Config	Notes
Cloud Run	min=0, max=10	Scales to zero overnight to minimize cost; cold start ~1.5s (acceptable for hobby project)
Cloud Run CPU	1 vCPU / 512 MB	Sufficient for Next.js SSR + API routes; upgrade path to 2 vCPU trivial
Artifact Registry	us-central1	Docker image repo; cleanup policy: keep last 5 tags to control storage cost
Cloud Storage Bucket	Standard, us-central1	fossil-images bucket; uniform bucket-level IAM; no public access; all access via signed URLs
Secret Manager	3 secrets	DATABASE_URL, MISTRAL_API_KEY, NEXTAUTH_SECRET; automatic version management
Cloud Build Trigger	Push to main	GitHub repo webhook; triggers on push to main branch; build timeout 10 minutes
Cloud Armor	OWASP preset	Attached to Load Balancer; rate limit 100 req/min per IP; geo-block as needed

8. Security Architecture

8.1 Security Design Principles

- Defence in depth: security controls at network, application, data, and storage layers.
- Zero secrets in code or environment files: all sensitive values in GCP Secret Manager, injected at runtime.
- Ownership enforcement: every database operation scoped by authenticated user_id no user can read or modify another user's fossils.
- No direct storage access: GCS images never publicly accessible; all served via short-lived signed URLs.
- Minimal attack surface: single containerized service with no unnecessary open ports or services.

8.2 Security Controls

Layer	Control	Implementation
Network	GCP Cloud Armor WAF	OWASP CRS ruleset; rate limiting 100 req/min/IP; SQL injection and XSS rule sets enabled
Network	HTTPS Only	Cloud Run enforces HTTPS; HTTP → HTTPS redirect at Load Balancer; HSTS header set
Network	Cloud Run Ingress	Ingress set to "Load Balancer only" direct Cloud Run URL blocked from public access
Identity	NextAuth.js Sessions	httpOnly JWT session cookies; Secure flag; SameSite=Lax; 30-day session expiry
Identity	Password Hashing	bcrypt with cost factor 12; no plaintext passwords stored; NIST-compliant minimum length 8
Identity	OAuth via Google	Optional Google Sign-In via NextAuth OAuth provider; no password stored for OAuth users
Authorization	Ownership Middleware	Every /api/fossils/[id] route verifies fossil.user_id === session.user.id before any operation
Authorization	Row-Level Isolation	All SQL queries include WHERE user_id = $session_user_id no cross-user data leakage possible
Data	NeonDB Encryption	Encryption at rest (AES-256) and in transit (TLS 1.3) managed by Neon
Data	GCS Signed URLs	Images served only via time-limited signed URLs (1-hour expiry); bucket has no public IAM binding
Data	Input Validation	Zod schema validation on all API route inputs; file type whitelist (jpg/png/webp); 10 MB size cap
Secrets	GCP Secret Manager	DATABASE_URL, MISTRAL_API_KEY, NEXTAUTH_SECRET injected as env vars at Cloud Run startup
Secrets	IAM Least Privilege	Cloud Run service account: Secret Manager accessor + GCS object creator/viewer only
Application	CSRF Protection	NextAuth.js CSRF token on all auth mutations; SameSite cookie attribute mitigates cross-site attacks

Application	Content Security Policy	CSP header set: default-src self; img-src self storage.googleapis.com; script-src self
Application	Dependency Scanning	Artifact Registry vulnerability scanning on each Docker image push; GH Dependabot on repo

8.3 GCP IAM Role Bindings (Least Privilege)

```
Cloud Run Service Account: fossilyte-sa@<project>.iam.gserviceaccount.com


roles/secretmanager.secretAccessor
  → Read: fossilyte-db-url, mistral-key, nextauth-secret (only these 3)


roles/storage.objectCreator
  → Write: gs://fossilyte-fossil-images/*


roles/storage.objectViewer
  → Read: gs://fossilyte-fossil-images/*
  → (for signed URL generation no public viewer binding on bucket)


NO roles/editor, NO roles/owner, NO roles/storage.admin
NO compute or networking permissions
```

9. Scalability & Availability

9.1 Scalability Design

Fossilyte's architecture is intentionally serverless-first. Every major component scales independently and automatically without manual intervention critical for a solo-maintained project with no operations team.

Component	Scaling Model	Limit	Bottleneck Risk
Cloud Run	Auto (0→N instances)	max-instances=10	Low concurrent request isolation
NeonDB	Serverless autoscale	1 GB storage free tier	Connection pool (max 100 conns use pooler)
GCS Image Storage	Infinite (object store)	Unlimited objects	None fully managed
Mistral AI API	Managed by Mistral	Rate limit: 100 req/min	Medium queue if high demand
Cloud CDN	Global edge cache	Unlimited	None static assets cached globally

9.2 Availability Design

Risk	Target SLA	Mitigation
Cloud Run instance failure	99.95% (GCP SLA)	Cloud Run auto-restarts failed instances; multiple instances during load
NeonDB unavailability	99.95% (Neon SLA)	Neon provides automatic failover; connection pooler handles transient errors
GCS unavailability	99.99% (Google SLA)	Multi-zone redundant by default; images served with 1-hour signed URL cache buffer
Mistral API outage	Best effort	Identification feature degrades gracefully; fossil CRUD unaffected; UI shows friendly error
Cold start latency	< 2s cold, < 200ms warm	min-instances=0 for cost; consider min-instances=1 if UX complaints; Cloud Run keeps warm for active apps
Overall application SLA	**99.5% target**	Composite of Cloud Run + NeonDB SLAs; sufficient for hobby/passion project context

9.3 Cost Optimization (Personal Expense Context)

As a self-funded passion project, cost discipline is a first-class engineering concern. The architecture is deliberately designed to minimize monthly spend while remaining capable of real user growth.

Resource	Est. Monthly Cost	Cost Control Mechanism
Cloud Run	~$0–5	min-instances=0 (scales to zero); pay only for requests processed
NeonDB	$0 (free tier)	Free tier: 0.5 GB storage, 1 compute unit; sufficient for early user base
GCS Storage	~$0.02/GB/mo	Image compression before upload; lifecycle policy to delete orphaned images
Mistral AI	~$0.15/1M tokens	Pixtral-12B pricing; identification feature rate-limited to 5/day per user on free tier
Cloud Build	$0 (free tier)	120 build-minutes/day free sufficient for personal project CI/CD cadence
Secret Manager	~$0.18/mo	$0.06/secret/month × 3 secrets
TOTAL ESTIMATE	**< $10/month**	For small-to-medium user base; scalable to $30–50/month at thousands of active users

10. Performance Targets

Metric	Target (p95)	Strategy
Dashboard initial load (warm)	< 1.5 s	Next.js SSR + React Server Components; image lazy loading
Dashboard initial load (cold start)	< 3.0 s	Acceptable for hobby project; mitigated by Cloud Run warm-up
API response (CRUD operations)	< 300 ms	NeonDB pooler; indexed queries; no N+1 queries via Drizzle ORM
Image upload to preview	< 2.0 s	Client-side resize before upload (< 2 MB target); direct GCS upload via signed URL
ML identification response	< 8.0 s	Mistral API latency; streamed response with loading skeleton in UI
Image gallery scroll (24 items)	< 200 ms render	Signed URLs pre-generated server-side; next.js Image component with lazy loading
Search/filter response	< 100 ms	Client-side filtering on loaded page; debounced API call for server-side search

11. Future Roadmap

Phase	Feature	Description
v1.1	Public Collection Profile	Optional shareable public URL for a collector's gallery read-only, curated subset.
v1.2	Collection Value Estimator	Integrate market price data (auction records) to provide estimated value per specimen.
v1.3	Fossil Map View	Plotly-based geographic map showing collection provenance where each fossil was found.
v2.0	Community Features	Optional collector community: follow other collectors, public specimen comments, fossil spotting feed.
v2.1	Mobile PWA	Progressive Web App manifest + service worker for installable mobile experience without app store.
v2.2	Fine-tuned ML Model	Fine-tune Mistral on labelled fossil dataset to improve identification accuracy beyond general-purpose vision model.
v3.0	Freemium Tier	Free tier: 50 fossils, 3 AI IDs/month. Pro tier ($4.99/mo): unlimited fossils, AI IDs, CSV export, public profile.

12. Appendix

A. Glossary

Term	Definition
NeonDB	Serverless PostgreSQL provider; scales compute to zero on inactivity and auto-scales on demand; designed for cost efficiency.
Cloud Run	GCP serverless container platform; runs Docker containers on-demand with automatic scaling including scale-to-zero.
Drizzle ORM	TypeScript-native, lightweight SQL query builder and schema migration tool; no heavy abstraction over raw SQL.
NextAuth.js	Authentication library for Next.js; handles sessions, OAuth providers, CSRF, and JWT management.
Pixtral-12B	Mistral AI's multimodal vision-language model capable of analyzing image content and generating structured text responses.
Signed URL	Time-limited, cryptographically signed URL granting temporary access to a private GCS object without exposing credentials.
SSR	Server-Side Rendering page HTML generated on the server per request; improves SEO and initial load performance.
CSP	Content Security Policy HTTP header that restricts which resources a browser may load, mitigating XSS attacks.
SaaS	Software as a Service web-hosted software accessed via browser without local installation or maintenance.

B. Tech Stack Summary

Category	Technology	Version / Notes
Runtime	Node.js	v20 LTS (Alpine base image)
Framework	Next.js	v14 App Router, Server Components, API Routes
Language	TypeScript	v5.x strict mode enabled
Styling	Tailwind CSS	v3.x JIT mode
Database	NeonDB PostgreSQL	PostgreSQL 16 compatible; serverless driver @neondatabase/serverless
ORM	Drizzle ORM	v0.30+ type-safe schema; drizzle-kit for migrations
Auth	NextAuth.js	v4.x Credentials + Google OAuth provider
Validation	Zod	v3.x API input validation schemas
ML / AI	Mistral AI SDK	@mistralai/mistralai pixtral-12b-2409 model

Container	Docker	Multi-stage build; linux/amd64; target image < 250 MB
Cloud	GCP	Cloud Run, GCS, Cloud Build, Artifact Registry, Secret Manager, Cloud Armor
Package Manager	pnpm	v8.x faster installs, stricter dependency resolution

C. Revision History

Version	Date	Author	Changes
0.1	Feb 2026	Solo Engineer	Initial draft architecture proposal
0.5	Mar 2026	Solo Engineer	ML module + security section added
0.9	Mar 2026	Solo Engineer	Docker + CI/CD pipeline finalized
1.0	Apr 2026	Solo Engineer	Final review; cost model + roadmap added